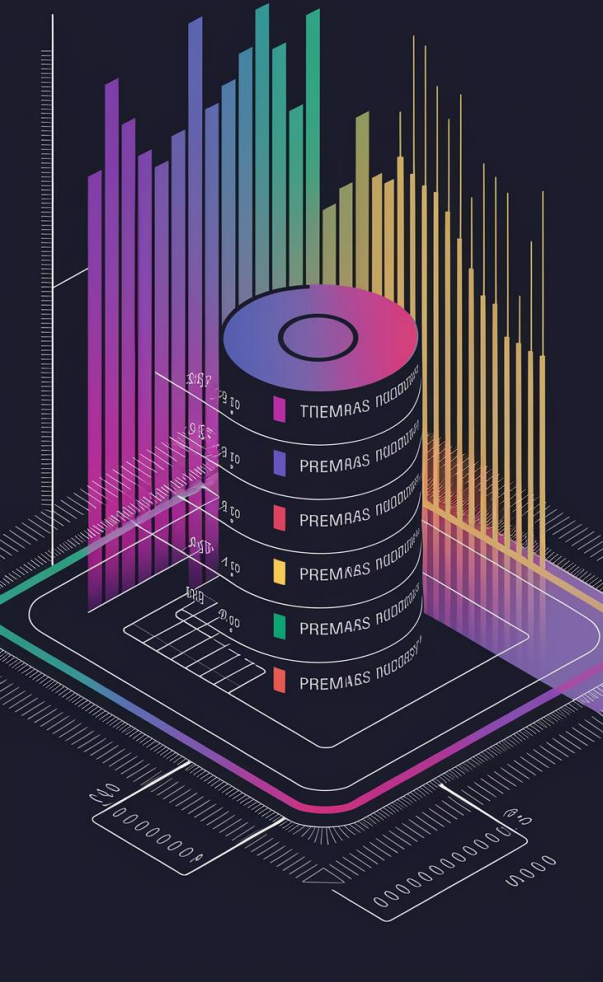




# Pruebas de Software: Pruebas de Rendimiento

**Semana 12**

**Ingeniería de Software 2**

**2026-1**



## Logro de la sesión



Los estudiantes comprenderán los conceptos fundamentales de las pruebas de rendimiento, incluyendo su importancia, los diferentes tipos de pruebas, el proceso para llevarlas a cabo y los conceptos clave de concurrencia, latencia y throughput, permitiéndoles analizar y evaluar el rendimiento de sistemas de software de manera efectiva.

- ¿Cómo podemos garantizar que un sistema de software funcione de manera eficiente bajo diferentes niveles de carga y condiciones extremas?
- ¿Qué impacto tienen la concurrencia, la latencia y el throughput en la experiencia del usuario y el rendimiento del sistema?
- ¿Por qué es importante realizar diferentes tipos de pruebas de rendimiento para asegurar la estabilidad y escalabilidad de una aplicación?
- ¿Qué pasos debemos seguir para identificar y resolver cuellos de botella en un sistema antes de su implementación en producción?



## Contenidos

- Introducción a las pruebas de rendimiento
- Tipos de Pruebas de Rendimiento
- Proceso de pruebas de rendimiento
- Concurrencia, Latencia y throughput

# Conceptos básicos de las pruebas de rendimiento y carga

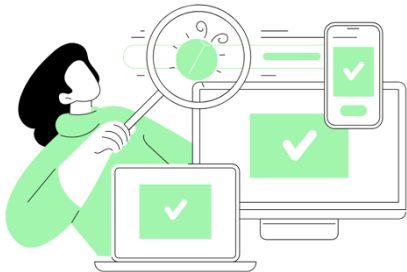
# Introducción a las pruebas de rendimiento

- Las pruebas de rendimiento son el proceso de determinar la velocidad o la eficacia de un ordenador, una red, un programa de software o un dispositivo.
- Antes de entrar en detalles, debemos entender los factores que rigen las pruebas de rendimiento:
  - Rendimiento
  - Tiempo de respuesta
  - Sintonización
  - Evaluación comparativa

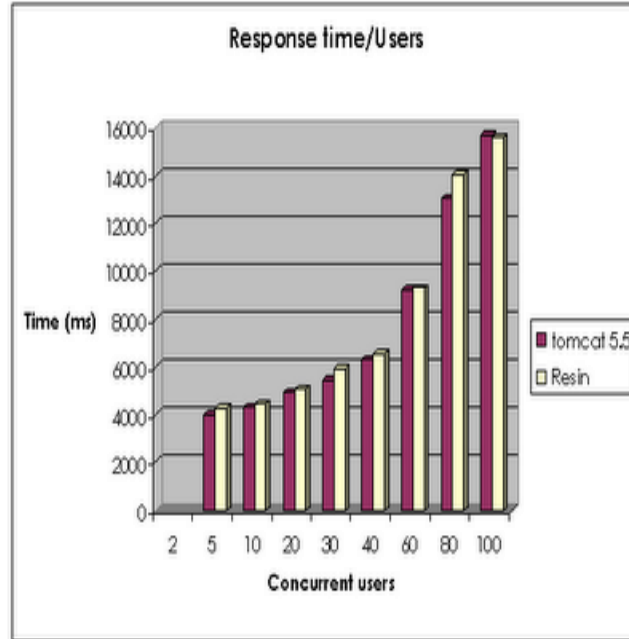


# Rendimiento

- Capacidad de un producto para manejar múltiples transacciones en un período determinado.
- El rendimiento representa el número de solicitudes/transacciones comerciales procesadas por el producto en un período de tiempo especificado.
- A medida que aumenta el número de usuarios simultáneos, el rendimiento aumenta casi linealmente con el número de solicitudes. Como hay muy poca congestión dentro de las colas del sistema del servidor de aplicaciones.



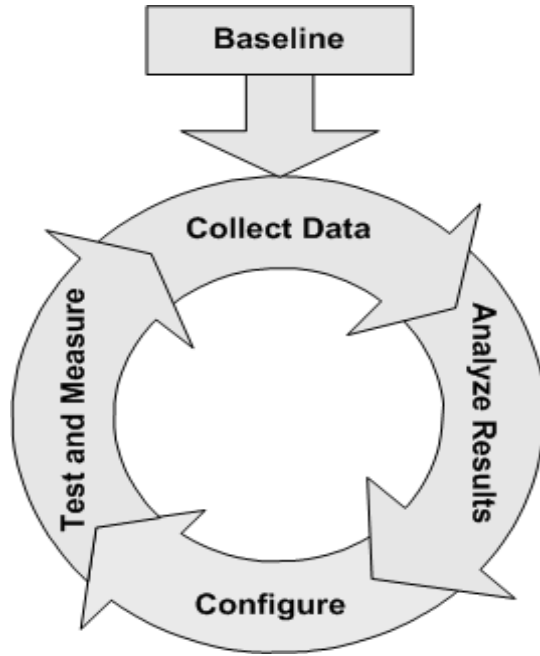
# Tiempo de respuesta



- Es igualmente importante averiguar cuánto tiempo tardó en completarse cada una de las transacciones.
- El tiempo de respuesta se define como el **retraso entre el punto de solicitud y la primera respuesta del producto.**
- El tiempo de respuesta aumenta proporcionalmente a la carga de usuarios.

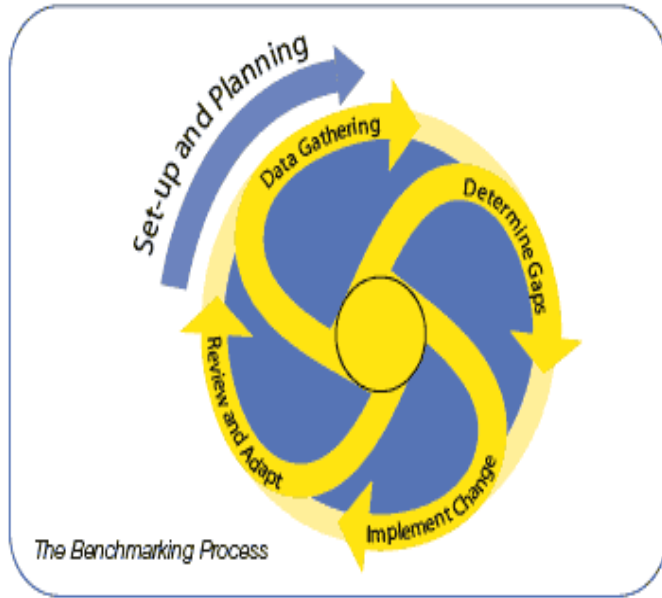


# Sintonización



- El ajuste es el procedimiento mediante el cual se mejora el rendimiento del producto mediante el establecimiento de diferentes valores para los parámetros del producto, el sistema operativo y otros componentes.
- El ajuste mejora el rendimiento del producto sin tener que tocar el código fuente del producto.

# Benchmarking



- Un rendimiento muy bien mejorado de un producto no tiene sentido desde el punto de vista comercial si ese rendimiento no coincide con los productos de la competencia.
- Es necesario un análisis cuidadoso para trazar la lista de transacciones que se compararán entre los productos, de modo que sea posible una comparación manzana-manzana.

# Pruebas de rendimiento: definición

- Son un proceso que define si el software cumple los requisitos de velocidad, escalabilidad y estabilidad en diferentes circunstancias, con esto no buscamos encontrar errores, sino identificar los cuellos de botella y eliminarlos para evitar fallos o demoras de la aplicación, optimizando y mejorando la aplicación previa al lanzamiento de la misma.
- Las pruebas de rendimiento se realizan para derivar números de referencia para el sistema.
- No se aplica carga pesada al sistema
- El ajuste se realiza hasta que el sistema bajo prueba alcanza los niveles esperados de rendimiento.

## Cuando usarla

Es importante considerar si las pruebas se realizarán en un software desarrollado desde cero o en uno ya existente. Esta distinción influye en la estrategia de pruebas, los recursos necesarios y los enfoques a seguir.

### Software Existente

Hay que considerar los beneficios de las pruebas ya que si hay que cambiar algo puede salir mas costoso realizarlos a ese punto y si se pueden realizar esos cambios

### Software Nuevo

Debemos considerar la metodología que usaremos para determinar en que momento aplicar las pruebas siendo de la siguiente forma:

- **Cascada:** Al final del desarrollo.
- **Ágil:** Al principio del desarrollo.

# Tipos de Pruebas de Rendimiento

## Pruebas de Carga

- Las pruebas de carga son aquellas que se enfocan en la habilidad de un sistema para gestionar niveles de carga (posibles en la realidad) de forma anticipada a que ocurran.
- Se realizan a través del incremento de solicitudes generadas por un número controlado de usuarios o procesos.

## Pruebas de Estrés

- Esta es una medida de la capacidad de un sistema o componente para manejar una carga máxima igual o superior al límite.
- Se puede utilizar para medir el rendimiento de un sistema o componente en particular cuando los recursos (como el ancho de banda, la CPU, la memoria, etc.) se están agotando.

## Pruebas de Capacidad

- pruebas de estrés. La carga en el sistema monitoreado aumenta bajo ciertas condiciones de falla.
- De esta manera, las pruebas de rendimiento pueden medir cuántos usuarios se pueden manejar sin sacrificar el rendimiento, o incluso sin violar los objetivos mínimos establecidos.

## Pueba de Escalabilidad

- En las pruebas de escalabilidad, el objetivo es medir si el sistema puede cumplir objetivos de rendimiento viables en el futuro.
- Gracias a estas pruebas, es posible determinar si el sistema puede crecer (aumentando el número de usuarios, conexiones concurrentes, etc.) sin errores o en línea con las expectativas planificadas.

# Tipos de Pruebas de Rendimiento

## Pruebas de Resistencia

- En este caso nos encontramos con un tipo de test que se centra en la estabilidad del sistema durante un determinado periodo de tiempo en un contexto particular del sistema operativo.
- Para una mejor comprensión, estas pruebas miden la capacidad del sistema para manejar escenarios específicos que demandan recursos, es decir, sin fugas de memoria, número de problemas de conexión a la base de datos, grupo de subprocesos, etc.

## Pruebas de Concurrencia

- En este caso nos encontramos con un tipo de test que se centra en la estabilidad del sistema durante un determinado periodo de tiempo en un contexto particular del sistema operativo.
- Para una mejor comprensión, estas pruebas miden la capacidad del sistema para manejar escenarios específicos que demandan recursos, es decir, sin fugas de memoria, número de problemas de conexión a la base de datos, grupo de subprocesos, etc.

## Spike testing

- En este tipo de pruebas, el proceso es un aumento o disminución repentina de la carga generada por una gran cantidad de usuarios mientras se monitorea el comportamiento del sistema.
- Con esta prueba, puede ver si los cambios de carga repentinos e inesperados tienen un impacto en el rendimiento.

# ¿Por qué las pruebas de rendimiento?

- Identifica los problemas desde el principio, antes de que sean costosos de resolver.
- Reduce los ciclos de desarrollo.
- Produce código de mejor calidad y más escalable.
- Evita la pérdida de ingresos y credibilidad debido a un rendimiento deficiente del sitio web.
- Permite una planificación inteligente para futuras expansiones.
- Para garantizar que el sistema cumpla con las expectativas de rendimiento, como el tiempo de respuesta, el rendimiento, etc., bajo niveles de carga determinados.
- Exponga errores que no aparecen en las pruebas superficiales, como errores de administración de memoria, pérdidas de memoria, desbordamientos de búfer, etc.



# ¿Cuándo se requiere?

## **Fase de diseño:**

Páginas que contienen muchas imágenes y multimedia para tiempos de espera razonables. Las cargas pesadas son menos importantes que saber qué tipos de contenido causan ralentizaciones.

## **Fase de desarrollo:**

Para comprobar los resultados de páginas y procesos individuales, buscando puntos de ruptura, código innecesario y cuellos de botella.

## **Fase de implementación:**

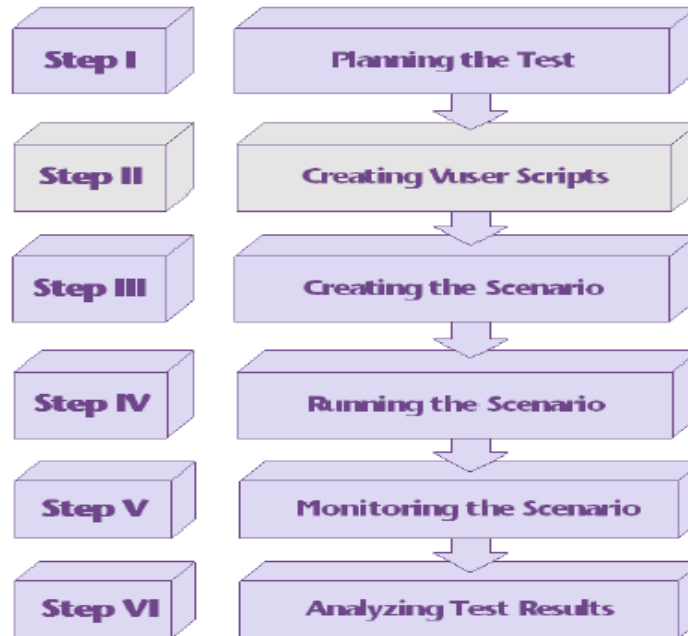
Identificar los requisitos mínimos de hardware y software para la aplicación.



# ¿Qué se debe probar?

- **Transacciones de alta frecuencia:** Las transacciones utilizadas con más frecuencia tienen el potencial de afectar el rendimiento de todas las demás transacciones si no son eficientes.
- **Transacciones de misión crítica:** Se deben incluir las transacciones más importantes que facilitan los objetivos centrales del sistema, ya que el fracaso bajo carga de estas transacciones tiene, por definición, el mayor impacto.
- **Transacciones de lectura:** Se debe incluir al menos una transacción de SOLO LECTURA, de modo que el rendimiento de dichas transacciones pueda diferenciarse de otras transacciones más complejas.
- **Transacciones de actualización:** Se debe incluir al menos una transacción de actualización para que el rendimiento de dichas transacciones se pueda diferenciar de otras transacciones.

# Proceso de pruebas de rendimiento



# 1. Planificación

- **Determinar los objetivos de las pruebas de rendimiento**
- **Describir la aplicación que se va a probar mediante un modelo de aplicación**

1. Describir el entorno de hardware

2. Crear un Punto de Referencia (Agenda) que se registrará en la Fase 2.

A. Definir qué tareas realizará cada usuario

B. Defina (o estime) el porcentaje de usuarios por tarea.

# Registro -> Modificación

## Registro

- Registre las actividades de prueba definidas que se usarán como base para los scripts de prueba de carga.
- Una actividad por tarea o varias actividades en función de la definición de la tarea del usuario

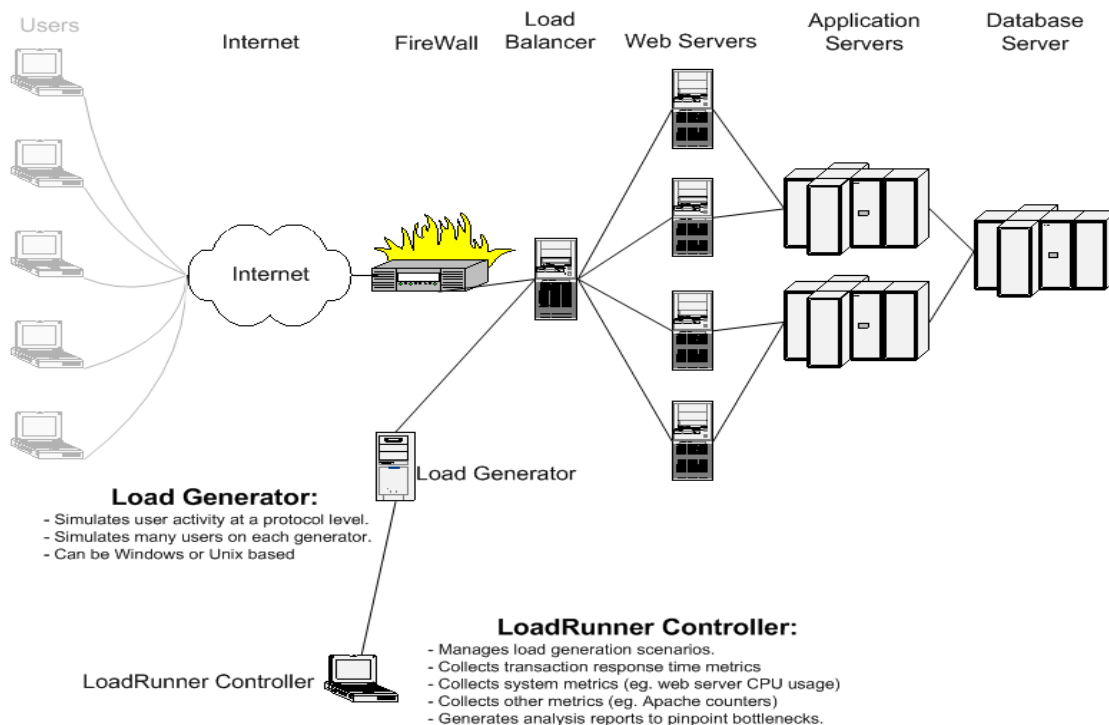
## Modificar

- Modifique los scripts de prueba de carga definidos por la grabadora para reflejar simulaciones de prueba de carga más realistas.
- Definición del proyecto, usuarios
- Aleatorizar parámetros (datos, tiempos, entorno)
- Aleatorizar las actividades de usuario que se producen durante la prueba de carga

## Supervisar -> Analizar

- **Supervisión del escenario:** Supervisamos la ejecución de escenarios utilizando los diversos monitores de tiempo de ejecución en línea
- **Análisis de los resultados de las pruebas:** Durante la ejecución del escenario, la herramienta registra el rendimiento de la aplicación bajo diferentes cargas. Utilizamos los gráficos e informes para analizar el rendimiento de la aplicación.

# Configuración de prueba de carga para un sistema web



# Herramientas Para Las Pruebas De Rendimiento



LoadView, desarrollado por Dotcom-Monitor, es una solución integral para realizar pruebas de rendimiento en sitios web, aplicaciones web, API y medios de streaming. Esta herramienta ofrece pruebas de rendimiento auténticas que se ejecutan en un entorno de navegador real, lo que permite simular condiciones de usuario reales y evaluar el comportamiento de las aplicaciones bajo diferentes cargas. Con LoadView, los usuarios pueden identificar cuellos de botella, optimizar el rendimiento y garantizar una experiencia fluida para los usuarios finales.



LoadRunner es una herramienta de pruebas de rendimiento desarrollada por Micro Focus desde 1993, diseñada para realizar pruebas de concurrencia al simular millones de usuarios virtuales. Disponible para Windows y Linux en 11 idiomas, permite la ejecución de scripts de Apache JMeter desde su versión 12.55. Con componentes como Virtual User Generator, Controller y Load Generators, LoadRunner ayuda a identificar cuellos de botella y optimizar el rendimiento de aplicaciones, integrándose también con herramientas de DevOps y APM para una gestión eficiente en entornos de desarrollo ágil.



Apache JMeter es una herramienta de código abierto desarrollada por la Apache Software Foundation, diseñada para realizar pruebas de rendimiento y carga en aplicaciones web, servicios web y bases de datos. Permite simular múltiples usuarios concurrentes y es altamente configurable, soportando diversos protocolos como HTTP, HTTPS, FTP y JDBC. Su interfaz gráfica facilita la creación de escenarios de carga complejos mediante la adición de grupos de usuarios, temporizadores y aserciones, además de ofrecer capacidades de análisis de resultados a través de gráficos y reportes detallados. JMeter es popular en entornos de desarrollo ágil y DevOps, gracias a su capacidad de integración con otras herramientas y su naturaleza personalizable.



Neoload también es software propietario, está desarrollado en Java, y disponible para Windows, Linux, MacOS y Solaris. Sus scripts se desarrollan a través de una interfaz de forma más intuitiva que con las anteriores herramientas. Destaca también por la velocidad de ejecución de sus pruebas.



# Herramienta de prueba de carga



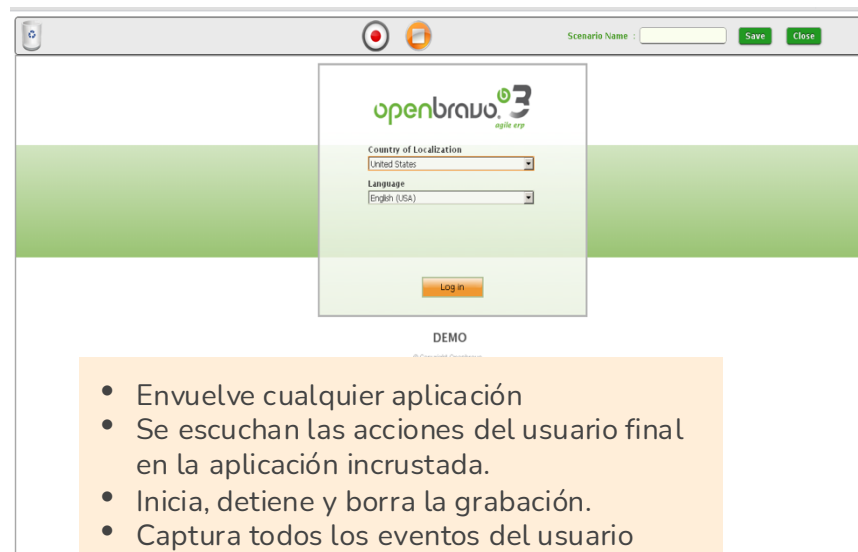
# Módulos de herramientas de prueba

- Grabadora
- Generador de escenarios
- Editor de parámetros de escenario
- Creador de casos de prueba
- Pruebas de ejecución
- Análisis de resultados

# Grabadora

- La grabadora se ejecuta en el servidor de proyectos
- Atiende las solicitudes de los clientes
- Inicia o detiene la grabación en cualquier momento
- Registra cualquier interacción del usuario con el servidor
- Restablece la grabación
- Representa la captura de pantalla en la aplicación cliente.
- Escucha las operaciones del usuario final en la aplicación y las guarda como escenario de prueba

# Generador de escenarios



# Editor de parámetros de escenario

- Enumera todos los escenarios que registra la grabadora de escenarios
- Muestra los parámetros del escenario que el usuario final publica en el servidor durante la interacción con la aplicación.
- Cualquier valor de parámetro puede ser modificado con el nuevo, además de esto, los nombres de los parámetros también pueden ser editados, considerando las transacciones en el sistema back-end, pueden requerir diferentes valores de entrada para evitar restricciones, por lo tanto, está disponible la entrada de múltiples valores para un solo parámetro, en cada ejecución se utiliza el valor del parámetro en la lista para realizar las transacciones con éxito.
- Cualquier escenario modificado se puede guardar como un nuevo escenario

Scenario Names

satis

Close Delete

New Scenario Name

Create

MODIFY SCENARIO

Filter Modified Fields

POSTING DATA

2

\_afriWindowId

null

+

\_afriLoop

4106431001690171

+

\_afriWindowMode

0

+

4

event.cb1

<cn xmlns="http://oracle.com/richClient/comm"><v="b

+

event

cb1

+

uname

telauro

+

\_adf.crl-state

16j0f77m0w\_4

+

org.apache.myfaces.trinidad.faces.FORM

f1

+

javax.faces.ViewState

1y006zccax

+

pass

ccl234

+

5

\_afriLoop

4106440361825580

+

\_adf.crl-state

16j0f77m0w\_4

+

10

unique

1305573873695

+

oracle.adf.view.rich.STREAM

pt1:pt\_t1

+

\_adf.crl-state

16j0f77m0w\_4

+

Adf-Rich-Message

true

+

oracle.adf.view.rich.forceHTML

true

+

javax.faces.ViewState

1y006zccay

+

35

# Entrada de valor de parámetro

Scenario Names

satis

Close

Delete

New Scenario Name

Create

MODIFY SCENARIO

☐ Filter Modified Fields

POSTING DATA

2

\_strWindowId

null

+

\_strLoop

4106431001690171

+

\_strWindowMode

0

+

4

event.cb1

<m xmlns="http://oracle.com/richClient/comm">ck v="b

+

event

cb12

+

telaurl

16j0f77m0w\_4

+

f1

lj00l6zcax

+

cc1234

5

+

4106440361825580

16j0f77m0w\_4

+

33

1305573873695

+

pt1:pt\_t1

16j0f77m0w\_4

+

Adf-Rich-Message

true

+

oracle.adf.view.rich.forceHTML

true

+

Main Page - Microsoft Internet Explorer provid...

http://212.253.125.121:3573/AppTestClient.jsp/ModifyScen

Input parameters for [satis.4\_event]

cb12

cb12

cb13

cb14

cb15

cb16

cb17

cb18

cb19

cb20

cb21

cb23

cb32

cb42

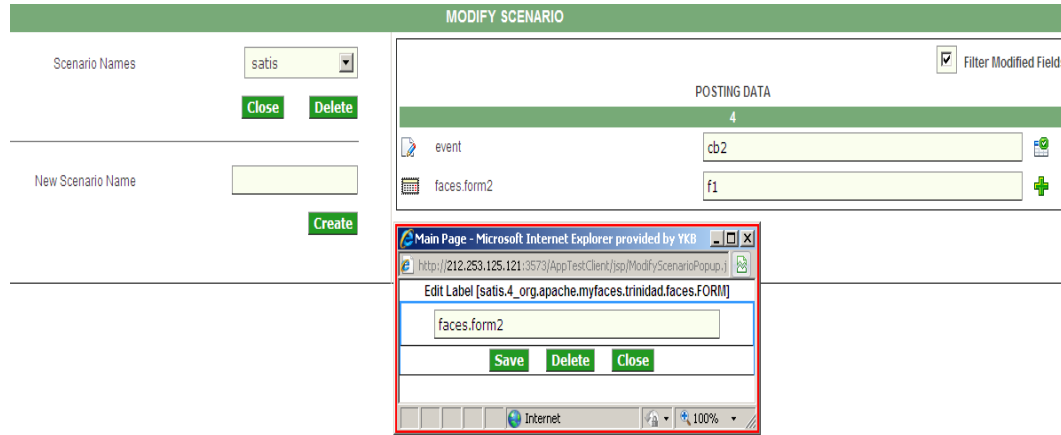
ch.4.2

Save

Delete

Close

# Modificación del nombre del parámetro



MODIFY SCENARIO

Scenario Names: satis [Close] [Delete]

New Scenario Name: [Create]

POSTING DATA

| 4           |     |
|-------------|-----|
| event       | cb2 |
| faces.form2 | f1  |

Filter Modified Fields: ☒

Modal: Edit Label [satis.4\_org.apache.myfaces.trinidad.faces.FORM]  
faces.form2 [Save] [Delete] [Close]

- Cualquier nombre de parámetro puede ser modificado, teniendo en cuenta las pruebas funcionales es importante enviar valores con diferentes nombres de parámetros.

# Creador de casos de prueba

Test Case Definition

Scenario Names

satis

Number Of Users

100

Test Repeat Count

1

Sleep Time Between Tests(ms)

10

Add To List

| Scenario   | Num Of Users | Repeat Count | Sleep Time(ms) |        |
|------------|--------------|--------------|----------------|--------|
| demo_sorgu | 10           | 1            | 0              | Remove |
| satis      | 100          | 1            | 10             | Remove |

Close

Test Case Name : tc\_test\_case1

Save Test Case

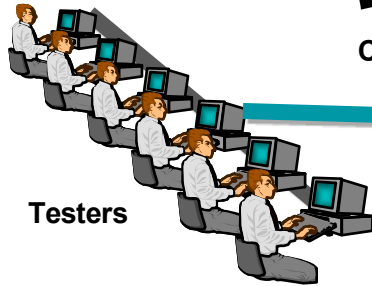
Se puede crear un caso de prueba mediante la combinación de escenarios  
Puede definir muchos escenarios para que se ejecuten en un solo caso de prueba.

# Load Generator

Todos ustedes,  
hagan clic en el  
botón GO  
nuevamente



Coordinador



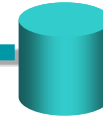
Testers

*Generación de carga*

Análisis?



Web server



Database server

*Sistema bajo prueba*

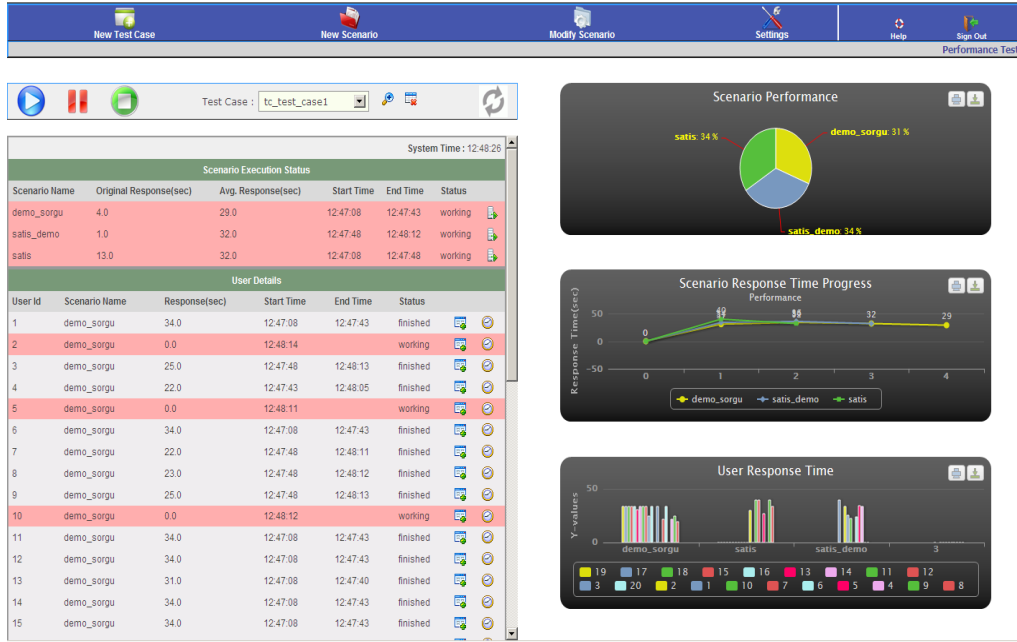
El Load Generator es un componente esencial en herramientas de pruebas de carga y rendimiento. Su función principal es generar la carga simulada sobre el sistema bajo prueba (SUT, por sus siglas en inglés: System Under Test) al ejecutar múltiples usuarios virtuales (Vusers) que realizan solicitudes o transacciones de manera simultánea. Este componente permite evaluar cómo un sistema responde bajo diferentes niveles de carga, desde condiciones normales hasta situaciones de estrés extremo.



## Load Generator

- Define la estrategia de ejecución del escenario
- Se puede ejecutar un escenario con  $n$  usuarios con  $n$  tiempo de repetición y  $n$  milisegundos de tiempo de suspensión entre las ejecuciones.
- Los escenarios se definen para trabajar juntos para formar un caso de prueba.
- Teniendo en cuenta la definición y combinación de ejecución de diferentes escenarios, se pueden crear muchos casos de prueba para ser ejecutados.

# Ejecución de pruebas

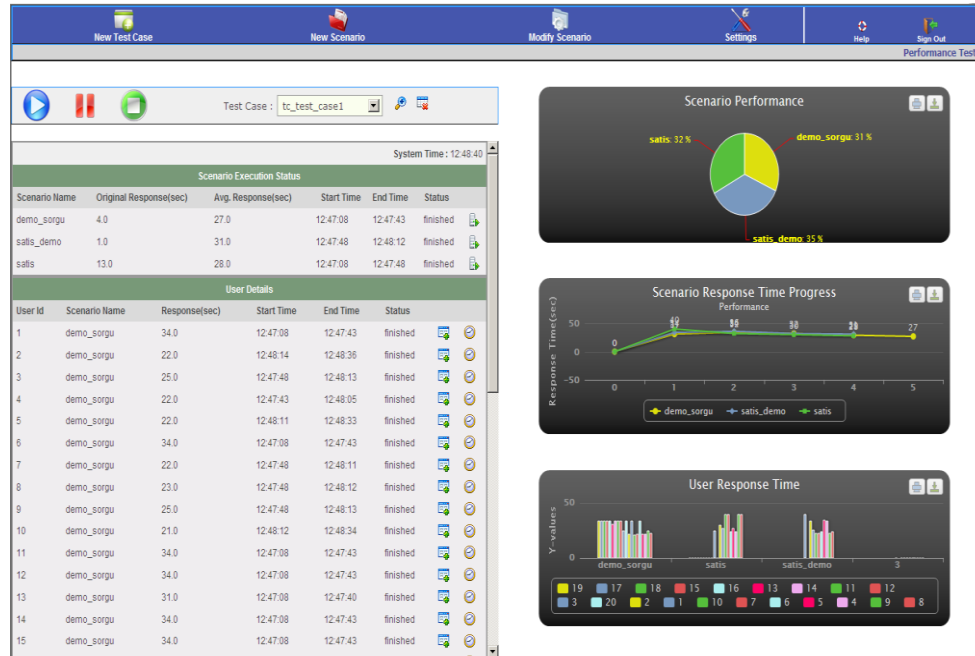


# Funciones del Load Generator

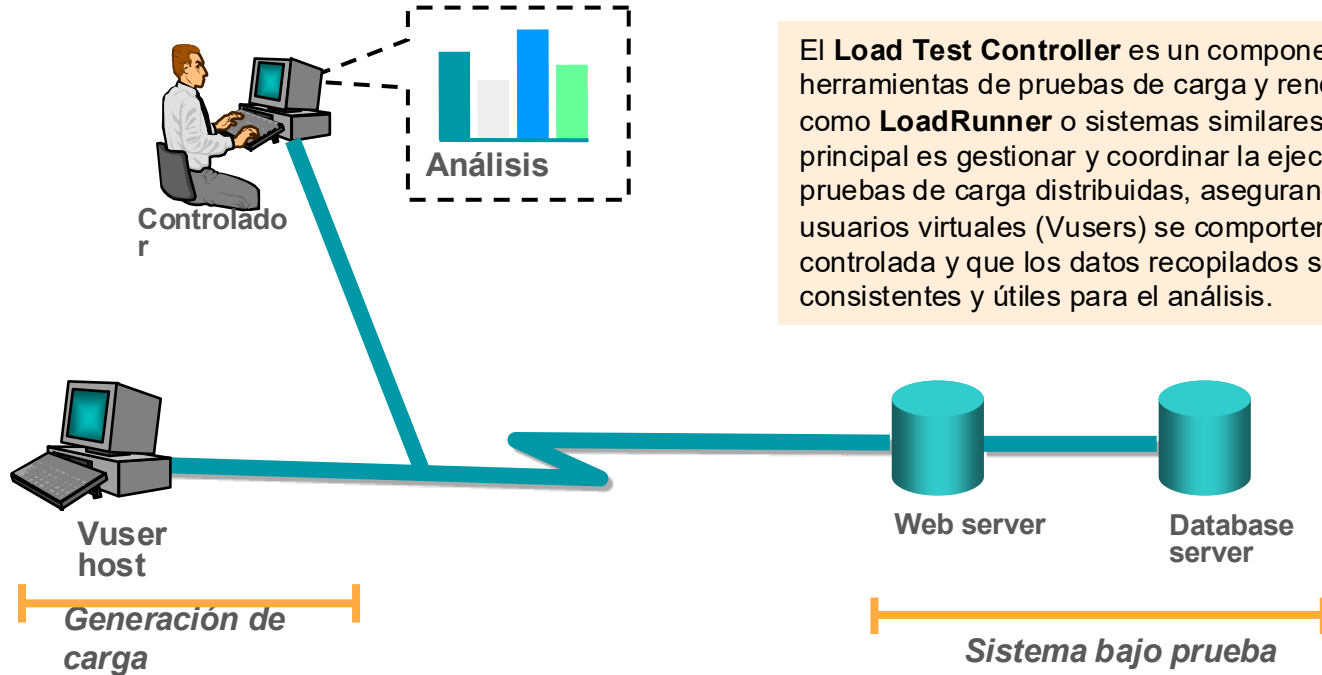
- **Generación de Carga:** Simula usuarios virtuales ejecutando scripts que replican el comportamiento de usuarios reales.
- **Distribución de la Carga:** Reparte la carga entre múltiples generadores para emular tráfico desde diferentes ubicaciones.
- **Ejecución de Escenarios:** Ejecuta patrones de carga como constante, incremental o picos de tráfico.
- **Recopilación de Métricas:** Registra datos clave como tiempos de respuesta, throughput y errores.
- **Escalabilidad:** Ajusta dinámicamente el número de usuarios virtuales para evaluar el sistema bajo diferentes niveles de carga.



# Analyzing Results



# Load Test Controller



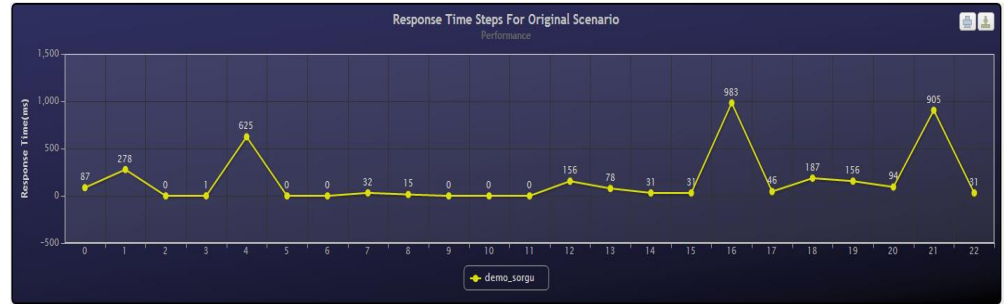
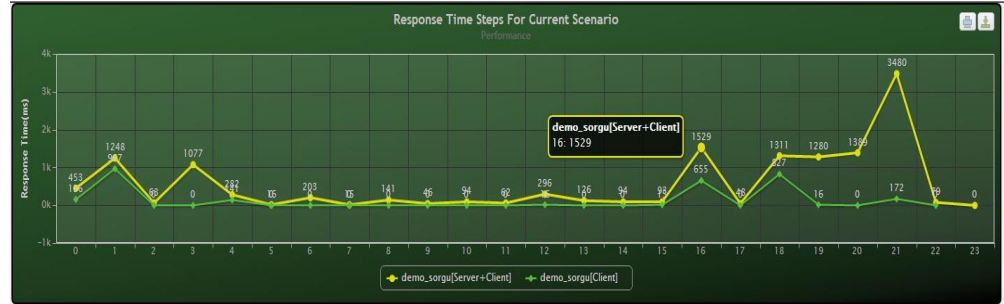
El **Load Test Controller** es un componente clave en herramientas de pruebas de carga y rendimiento, como **LoadRunner** o sistemas similares. Su función principal es gestionar y coordinar la ejecución de pruebas de carga distribuidas, asegurando que los usuarios virtuales (Vusers) se comporten de manera controlada y que los datos recopilados sean consistentes y útiles para el análisis.

# Funciones Principales del Load Test Controller

- **Gestión de Escenarios de Prueba:** Diseña, configura y ejecuta escenarios de carga, definiendo parámetros como usuarios virtuales, duración y patrones de carga (constante, incremental, picos).
- **Coordinación de Generadores de Carga:** Distribuye la carga entre generadores para simular usuarios desde distintas ubicaciones, asegurando sincronización.
- **Monitoreo en Tiempo Real:** Proporciona métricas como tiempos de respuesta, throughput y uso de recursos (CPU, memoria, red), identificando problemas durante la prueba.
- **Control de Usuarios Virtuales (Vusers):** Gestiona el número de usuarios virtuales y simula comportamientos como inicios de sesión, navegación o transacciones.
- **Recopilación de Datos:** Centraliza y asegura la consistencia de los datos para análisis posterior del rendimiento.
- **Configuración de Escenarios Avanzados:** Permite pruebas complejas con múltiples scripts, cargas distribuidas geográficamente y pruebas de estrés.

# Ejecución de escenarios de usuario

- La ejecución del escenario de usuario se muestra como un gráfico.
- El gráfico muestra la interacción del usuario con el servidor para realizar un único escenario.
- El eje derecho muestra los pasos de ejecución, mientras que el izquierdo imprime el tiempo de ejecución de cada paso en milisegundos.
- Las líneas amarillas muestran el tiempo de ejecución que transcurre en el servidor.
- Las líneas verdes muestran el tiempo de ejecución que pasa en el lado del cliente, en otras palabras, es el tiempo de representación de datos, si la red es lenta o la cantidad de datos es enorme, los tiempos de ejecución de la línea verde serán más altos en consecuencia.
- Todos los gráficos se pueden imprimir.



- demo\_sorgu(current)**

| Content Type           | Content Length(bytes) | Server Response Date | Client Rander Time(ms) |
|------------------------|-----------------------|----------------------|------------------------|
| text/xml;charset=utf-8 | 61999                 | 17:05:2011 08:25:25  | 855                    |

16 / 22    Response Time(ms.): 1520    ☐ Enable Java Script

<<   F5E   NEXT >>

| POSTED VALUES                                              |                                                                                                                                                                                  |           |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Request Parameters                                         | Request Headers                                                                                                                                                                  | Form Data |
| oracle.adf.view.rich.DELTAS {pt1_pt_11=-_afLocalDisState=} | ad-fsch-message true                                                                                                                                                             |           |
| event pt1_pt_11_22.c1                                      | Cookie JSESSIONID=01W1KTSGBK5ZyYVQIHQMlTJcpQq1RkH2v6qCST0DZFJBLmP-VQI-247038870; path=/; HttpOnly                                                                                |           |
| _adf.ctx.state 5d2r6b5yu_8                                 | Path Translated null                                                                                                                                                             |           |
| event.pt1_pt_11_22.c1 action                               | ad-fsch-page-id 1                                                                                                                                                                |           |
| org.apache.myfacesinidad.faces.FORM f1                     | Method POST                                                                                                                                                                      |           |
| java.faces.ViewState jsapqrpf54                            | RequestedSessionId 01W1KTSGBK5ZyYVQIHQMlTJcpQq1RkH2v6qDdt032ZFJbL1n-PVQI-2470388701305509921352                                                                                  |           |
|                                                            | User-Agent Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.0; Trident/4.0; SLCC1; .NET CLR 2.0.50727; Media Center PC 5.0; .NET CLR 3.5.30729; .NET CLR 3.0.30729; WYYTiemz2.NET |           |

|                         |                       |                       |
|-------------------------|-----------------------|-----------------------|
| *Müşteri Numarası       | Ctrl+Home             | *Müşteri Numarası     |
| Büro Kodu Kartı         | Ctrl+End              | Parti Numarası        |
| Gözetim Kartı           | Ctrl+End              | Kimlik Numarası       |
| Sokak Yüzeyi Sıra       | Isim                  | Isim                  |
| "Isim                   | İkinci Isim           | İkinci Isim           |
| "İkinci Isim            | Soyisim               | Soyisim               |
| **Soyisim               | Hesap Numarası        | Hesap Numarası        |
| **Hesap Numarası        | Eski Müşteri Numarası | Eski Müşteri Numarası |
| **Eski Müşteri Numarası | *Kriter Tipi          | *Kriter Tipi          |
| *Kriter Tipi            | Kriter Değeri         | Kriter Değeri         |





# Configuración

**Application Settings**

|                            |                                                                         |                                                                                                                                                            |
|----------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server Application URL     | <input type="text" value="http://10.115.226.139:7007/MTS/faces/index"/> | (The remote server url on which the performance tests are performed.)                                                                                      |
| Server Root Dir URL        | <input type="text" value="http://10.115.226.139:7007"/>                 | (The remote server base url adress)                                                                                                                        |
| Server Application Context | <input type="text" value="/MTS/"/>                                      | (The remote server context path)                                                                                                                           |
| Exception Key Words        | <input type="text" value="&lt;error"/>                                  | (When the remote server responses a request, the keywords will decide whether it completes the operation successfully or not. Put comma between keywords.) |

- La información de URL del servidor de aplicaciones puede definirse
- Teniendo en cuenta el html de respuesta del servidor, se puede definir cualquier estructura de error para decidir si la ejecución produjo un error o no.
- Teniendo en cuenta el estado de ejecución del escenario, esto puede ayudar a decidir sobre los casos de error.

# JMeter

JMeter es un programa Java diseñado para hacer pruebas de carga y rendimiento, Para poder determinar la velocidad con la que el sistema, bajo pruebas, realiza una tarea en las condiciones particulares del escenario de pruebas. Porque ayuda a detectar los cuellos de botella en las aplicaciones de software, antes que los usuarios sufran de un mal rendimiento además de una posible pérdida económica y la frustración de los usuarios

Prueba el rendimiento de servidores.

Permite realizar pruebas de rendimiento de distintos tipos de servidores:

- HTTP, HTTPS
- SOAP
- Bases de datos mediante JDBC
- JMS
- POP3 e IMAP.
- Servidores FTP.



# Concurrencia, Latencia y throughput

En el contexto de pruebas de rendimiento, los términos **concurrencia**, **latencia** y **throughput** son fundamentales para evaluar el comportamiento de una aplicación o sistema bajo carga. A continuación, se explican brevemente:

- **Concurrencia:** Se refiere al número de usuarios o procesos que interactúan simultáneamente con el sistema. En pruebas de rendimiento, la concurrencia mide cuántos usuarios virtuales están activos al mismo tiempo, ejecutando solicitudes o tareas. Es clave para evaluar cómo el sistema maneja múltiples usuarios simultáneos.
- **Latencia:** Es el tiempo que transcurre desde que un usuario envía una solicitud hasta que el sistema responde por primera vez. Representa el retraso inicial en la comunicación y es un indicador importante de la rapidez con la que el sistema comienza a procesar una solicitud.
- **Throughput:** Es la cantidad de datos o solicitudes que el sistema puede procesar en un período de tiempo determinado, generalmente medido en transacciones por segundo (TPS) o solicitudes por segundo. Es un indicador del rendimiento general del sistema y su capacidad para manejar grandes volúmenes de tráfico.



# Concurrencia

En pruebas de rendimiento, la concurrencia se mide simulando usuarios virtuales que realizan acciones específicas en el sistema. Herramientas como Apache JMeter, LoadRunner o Gatling permiten configurar escenarios de prueba con diferentes niveles de concurrencia. Los pasos típicos para medir la concurrencia incluyen:

- **Definir el escenario de prueba:** Identificar las acciones que los usuarios realizarán (por ejemplo, iniciar sesión, buscar productos, realizar compras).
- **Configurar el número de usuarios concurrentes:** Establecer cuántos usuarios virtuales interactuarán con el sistema al mismo tiempo.
- **Ejecutar la prueba:** Simular la carga concurrente y monitorear el comportamiento del sistema.
- **Analizar los resultados:** Evaluar métricas como tiempos de respuesta, tasas de error y throughput para determinar cómo el sistema maneja la concurrencia.



# Factores que Afectan la Concurrencia

Varios factores pueden influir en la capacidad de un sistema para manejar usuarios concurrentes:

- **Hardware:** La cantidad de CPU, memoria y capacidad de red disponibles afecta directamente la capacidad de concurrencia.
- **Arquitectura del sistema:** Sistemas bien diseñados, con balanceo de carga y escalabilidad horizontal, pueden manejar mayores niveles de concurrencia.
- **Bases de datos:** Las consultas lentas o bloqueos en la base de datos pueden limitar la concurrencia.
- **Configuración del servidor:** Parámetros como el número máximo de conexiones simultáneas permitidas en un servidor web o de aplicaciones pueden restringir la concurrencia.



# Buenas Prácticas para Manejar la Concurrency

- **Optimización del código:** Reducir operaciones costosas y mejorar la eficiencia del código para manejar múltiples solicitudes.
- **Uso de caché:** Implementar caché para reducir la carga en la base de datos y mejorar los tiempos de respuesta.
- **Escalabilidad horizontal:** Agregar más servidores para distribuir la carga entre ellos.
- **Pruebas regulares:** Realizar pruebas de concurrency periódicas para identificar problemas antes de que afecten a los usuarios finales.
- **Monitoreo en tiempo real:** Usar herramientas de monitoreo para detectar y resolver problemas de concurrency en producción.



# Concurrencia en Herramientas de Pruebas



- **Apache JMeter:** Permite configurar grupos de usuarios (Thread Groups) para simular múltiples usuarios concurrentes. Se pueden ajustar parámetros como el número de usuarios, el tiempo de inicio y la duración de la prueba.



- **LoadRunner:** Ofrece capacidades avanzadas para simular usuarios concurrentes y analizar el impacto en el sistema.

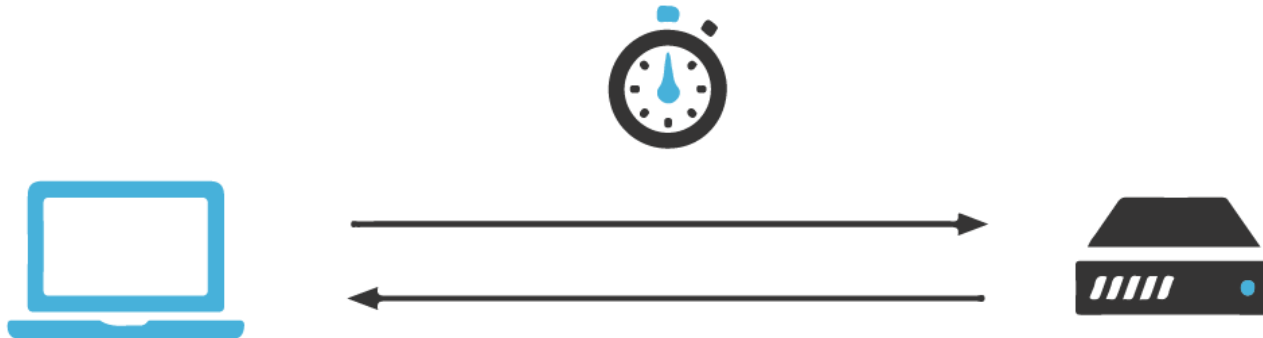


- **Gatling:** Proporciona un enfoque basado en scripts para simular cargas concurrentes y generar reportes detallados.

# Latencia

La latencia se mide en unidades de tiempo, generalmente en milisegundos (ms), y representa el tiempo que tarda un sistema en responder a una acción. Este tiempo incluye varios factores, como el tiempo de transmisión de datos, el procesamiento en el servidor y la entrega de la respuesta al cliente.

Por ejemplo, cuando un usuario hace clic en un enlace en un sitio web, la latencia es el tiempo que tarda el servidor en procesar la solicitud y enviar la primera parte de la respuesta al navegador del usuario.





# Componentes de la Latencia

|                                |                                                                                                                                                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latencia de Red                | <ul style="list-style-type: none"><li>• Tiempo que tarda un paquete de datos en viajar desde el cliente (usuario o dispositivo) al servidor y viceversa.</li></ul>                |
| Latencia de Procesamiento      | <ul style="list-style-type: none"><li>• Tiempo que tarda el servidor en procesar la solicitud del cliente.</li></ul>                                                              |
| Latencia de Transmisión        | <ul style="list-style-type: none"><li>• Tiempo necesario para transferir los datos desde el servidor al cliente.</li></ul>                                                        |
| Latencia del Cliente           | <ul style="list-style-type: none"><li>• Tiempo que tarda el dispositivo del cliente en procesar y renderizar la respuesta recibida.</li></ul>                                     |
| Latencia de Enlace             | <ul style="list-style-type: none"><li>• Tiempo que tarda en establecerse la conexión inicial entre el cliente y el servidor.</li></ul>                                            |
| Latencia de Retransmisión      | <ul style="list-style-type: none"><li>• Tiempo adicional causado por la necesidad de retransmitir paquetes de datos que se han perdido o dañado durante la transmisión.</li></ul> |
| Latencia de Capa de Aplicación | <ul style="list-style-type: none"><li>• Tiempo que tarda la aplicación en procesar la solicitud y generar una respuesta.</li></ul>                                                |

# Importancia de la Latencia

La latencia es un factor crítico en la experiencia del usuario y en el rendimiento de los sistemas. Una latencia alta puede provocar:

- **Experiencia de usuario negativa:** Los usuarios tienden a abandonar aplicaciones o sitios web que tardan demasiado en responder.
- **Impacto en aplicaciones en tiempo real:** En aplicaciones como videollamadas, juegos en línea o sistemas financieros, incluso pequeñas demoras pueden ser inaceptables.
- **Reducción de la productividad:** En entornos empresariales, una alta latencia puede ralentizar los flujos de trabajo y afectar la eficiencia.

# Cómo medir la Latencia

La latencia se mide utilizando herramientas y técnicas específicas que evalúan el tiempo de respuesta de un sistema. Algunos métodos comunes incluyen:

- **Ping:** Es una herramienta básica que mide la latencia de red enviando paquetes ICMP (Internet Control Message Protocol) a un servidor y midiendo el tiempo que tarda en recibir una respuesta.
- **Herramientas de pruebas de rendimiento:** Herramientas como Apache JMeter, LoadRunner o Gatling permiten medir la latencia en aplicaciones web y sistemas complejos. Estas herramientas generan solicitudes simuladas y registran el tiempo que tarda el sistema en responder.
- **Monitoreo en tiempo real:** Herramientas de monitoreo como New Relic, Dynatrace o AppDynamics pueden rastrear la latencia en tiempo real, proporcionando información detallada sobre las causas de los retrasos.
- **Pruebas de velocidad de red:** Servicios como Speedtest miden la latencia de red junto con el ancho de banda y la velocidad de descarga/carga.



# Factores que afectan la latencia

La latencia puede verse influenciada por diversos factores, entre los que destacan:

- **Distancia física:** Cuanto mayor sea la distancia entre el cliente y el servidor, mayor será la latencia debido al tiempo que tardan los datos en viajar.
- **Congestión de red:** Una red sobrecargada puede aumentar la latencia debido a la necesidad de reenviar paquetes o esperar en colas.
- **Capacidad del servidor:** Servidores con recursos limitados (CPU, memoria) pueden tardar más en procesar solicitudes, aumentando la latencia.
- **Tamaño de los datos:** Solicitudes o respuestas más grandes requieren más tiempo para transmitirse, lo que incrementa la latencia.
- **Configuración del sistema:** Configuraciones ineficientes en servidores, bases de datos o aplicaciones pueden introducir demoras innecesarias.
- **Proveedores de servicios de Internet (ISP):** La calidad del servicio proporcionado por el ISP del cliente o del servidor puede afectar significativamente la latencia.



# Reducir la Latencia

Reducir la latencia es esencial para mejorar el rendimiento del sistema y la experiencia del usuario. Algunas estrategias incluyen:

- **Optimización de la red:**
  - Usar redes de entrega de contenido (CDN) para acercar los datos al usuario final.
  - Implementar balanceadores de carga para distribuir el tráfico entre múltiples servidores.
- **Optimización del servidor:**
  - Mejorar la capacidad del servidor mediante actualizaciones de hardware o software.
  - Reducir la complejidad de las consultas a la base de datos y optimizar el código de la aplicación.
- **Compresión de datos:**
  - Comprimir las respuestas del servidor para reducir el tamaño de los datos transmitidos.
- **Uso de caché:**
  - Implementar caché en el cliente, servidor o CDN para evitar solicitudes repetitivas.
- **Minimización de solicitudes:**
  - Reducir el número de solicitudes necesarias para cargar una página o completar una acción.
- **Monitoreo y análisis continuo:**
  - Usar herramientas de monitoreo para identificar y resolver problemas de latencia en tiempo real.



# Throughput

El throughput se define como la cantidad de trabajo completado por un sistema en un intervalo de tiempo. Se mide en unidades como:

- **Solicitudes por segundo (RPS)**: En aplicaciones web, indica cuántas solicitudes HTTP puede procesar el servidor por segundo.
- **Transacciones por segundo (TPS)**: En sistemas de bases de datos, mide cuántas transacciones completas se ejecutan por segundo.
- **Bytes por segundo (Bps)**: En redes, mide la cantidad de datos transmitidos por segundo.
- **Operaciones por segundo (OPS)**: En sistemas de almacenamiento, mide cuántas operaciones de lectura/escritura se realizan por segundo.

Por ejemplo, si un servidor web procesa 500 solicitudes en 10 segundos, su throughput es de 50 solicitudes por segundo.



# Importancia del Throughput

El throughput es una métrica esencial porque:

- **Evalúa la capacidad del sistema:**
  - Permite determinar si un sistema puede manejar la carga esperada de usuarios o procesos concurrentes.
  - Es crucial para aplicaciones de alta demanda, como plataformas de comercio electrónico, servicios de streaming o sistemas financieros.
- **Identifica cuellos de botella:**
  - Un throughput bajo puede indicar problemas en la red, el servidor, la base de datos o el diseño de la aplicación.
- **Optimiza la experiencia del usuario:**
  - Un throughput alto asegura que los usuarios experimenten tiempos de respuesta rápidos y un servicio fluido.
- **Ayuda en la planificación de capacidad:**
  - Permite prever la infraestructura necesaria para soportar un crecimiento en la carga de trabajo.



# Factores que Afectan el Throughput

El throughput de un sistema puede verse influenciado por diversos factores, que incluyen:

- **Capacidad del Hardware:** CPU, memoria (RAM), disco, entre otros.
- **Capacidad de la Red:** Ancho de banda, latencia de red, pérdida de paquetes
- .
- **Diseño de la Aplicación:** Eficiencia del código, uso de recursos compartidos, bloqueos y concurrencia.
- **Carga del Sistema:** Número de usuarios concurrentes, tamaño de las solicitudes.
- **Configuración del Sistema:** Configuración del servidor, bases de datos.
- **Middleware y Servicios Externos:** APIs externas, middleware.



# Medir el Throughput

El throughput se mide utilizando herramientas y técnicas específicas que evalúan la capacidad del sistema para procesar solicitudes o datos. Algunos métodos comunes incluyen:

- **Pruebas de Carga**
- **Monitoreo en Tiempo Real**
- **Pruebas de Red**
- **Pruebas de Bases de Datos**

Fórmula Básica para Calcular el Throughput:

$$\text{Throughput} = \frac{\text{Número de solicitudes completadas}}{\text{Tiempo total (en segundos)}}$$

Por ejemplo, si un servidor procesa 1,000 solicitudes en 20 segundos:

$$\text{Throughput} = \frac{1,000}{20} = 50 \text{ solicitudes/segundo}$$



# Mejorar el Throughput

- **Optimización del Hardware**
- **Optimización de la Red**
- **Optimización del Código**
- **Uso de Caché**
- **Escalabilidad**
- **Optimización de Middleware**



## Ejercicio de Estudio



### **Ejercicio 1 :**

Un servidor web procesa 12,000 solicitudes HTTP en un período de 5 minutos.  
Calcula el throughput en términos de solicitudes por segundo (RPS).



### Solución :

Un servidor web procesa 12,000 solicitudes HTTP en un período de 5 minutos.  
Calcula el throughput en términos de solicitudes por segundo (RPS).

Número de solicitudes procesadas: 12,000

Tiempo total: 5 minutos =  $5 \times 60 = 300$  segundos

$$\text{Throughput} = \frac{\text{Número de solicitudes procesadas}}{\text{Tiempo total (en segundos)}}$$

$$\text{Throughput} = \frac{12,000}{300} = 40 \text{ solicitudes/segundo}$$

El throughput del servidor es de **40 solicitudes por segundo**.



## Ejercicio 2 :

Una base de datos procesa 18,000 transacciones en 30 minutos. Sin embargo, debido a problemas de rendimiento, solo logra completar 15,000 transacciones en ese tiempo. Calcula el throughput real en transacciones por segundo (TPS) y analiza la eficiencia del sistema.

### **Solución :**

Transacciones completadas: 15,000

Tiempo total: 30 minutos =  $30 \times 60 = 1,800$  segundos

$$\text{Throughput} = \frac{\text{Número de solicitudes procesadas}}{\text{Tiempo total (en segundos)}}$$

$$\text{Throughput} = \frac{15,000}{1,800} = 8.33 \text{ transacciones/segundo}$$

El sistema estaba diseñado para procesar 18,000 transacciones en 30 minutos, lo que equivale a un throughput esperado de:

$$\text{Throughput esperado} = \frac{18,000}{1,800} = 10 \text{ transacciones/segundo}$$



## Solución :

El sistema estaba diseñado para procesar 18,000 transacciones en 30 minutos, lo que equivale a un throughput esperado de:

$$\text{Throughput esperado} = \frac{18,000}{1,800} = 10 \text{ transacciones/segundo}$$

La eficiencia del sistema se calcula como:

$$\text{Eficiencia} = \frac{\text{Throughput real}}{\text{Throughput esperado}} \times 100$$

$$\text{Eficiencia} = \frac{8.33}{10} \times 100 = 83.3\%$$

El throughput real de la base de datos es de 8.33 transacciones por segundo, y la eficiencia del sistema es del **83.3%**.





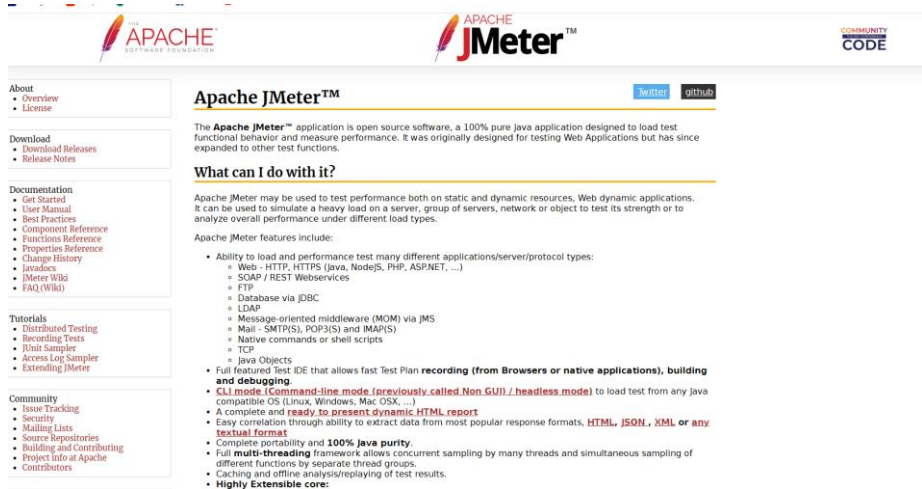
### Ejercicio 3 :

**Realice las siguientes actividades para llevar a cabo la prueba de carga utilizando la herramienta JMeter. A continuación, se detallan los pasos:**

- **Instalación de Jmeter**
- **Prueba de carga básica.**
- **Aplicación al proyecto del curso** :Siga estas indicaciones y aplíquelas al proyecto correspondiente al curso, adaptando la configuración de la prueba de carga a los requisitos específicos de su sistema o aplicación.

# Instalación

Para descargar JMeter accedemos a la siguiente dirección <http://jmeter.apache.org/> y accedemos en Dowload Releases.



## KEYS

### Apache JMeter 5.6.3 (Requires Java 8+)

#### Binaries

[apache-jmeter-5.6.3.tgz sha512 pgp](#)  
[apache-jmeter-5.6.3.zip sha512 pgp](#)

#### Source

[apache-jmeter-5.6.3\\_src.tgz sha512 pgp](#)  
[apache-jmeter-5.6.3\\_src.zip sha512 pgp](#)

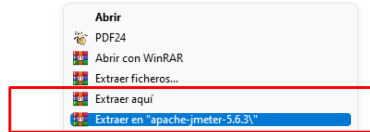
#### Archives



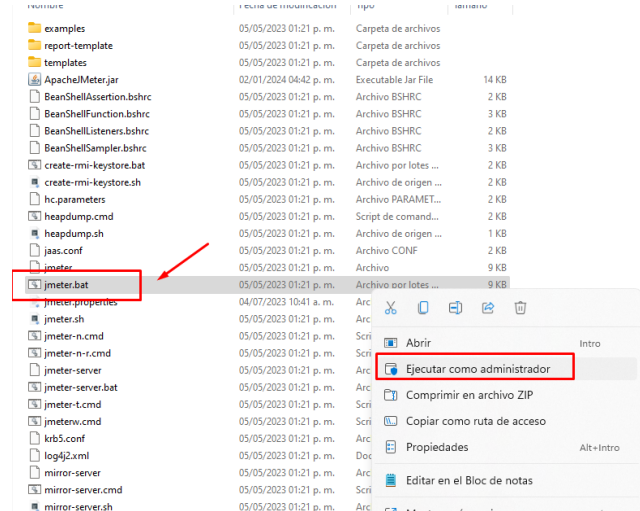
# Instalación

- Una vez que contamos con el archivo .zip se descomprime, de preferencia guardarlo directamente en el disco C.

| nombre                  | fecha de modificación  | tipo                | tamaño    |
|-------------------------|------------------------|---------------------|-----------|
| apache-jmeter-5.6.3     | 14/03/2025 12:42 p. m. | Carpeta de archivos |           |
| apache-jmeter-5.6.3.zip | 14/03/2025 12:41 p. m. | Archivo WinRAR Z... | 88,508 KB |

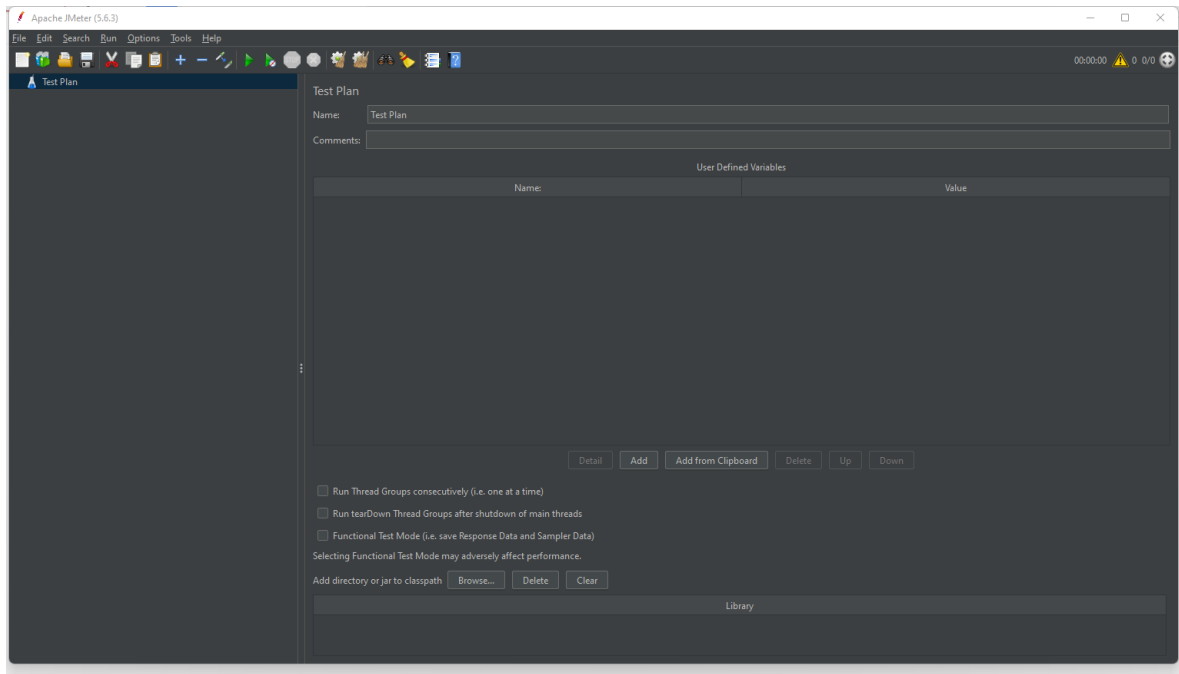


- Una vez descomprimidos obtendremos una carpeta apache-jmeter-5.6.3, dentro accederemos a la carpeta /bin donde encontraremos el archivo ejecutable jmeter.bat que será ejecutado en modo administrador.



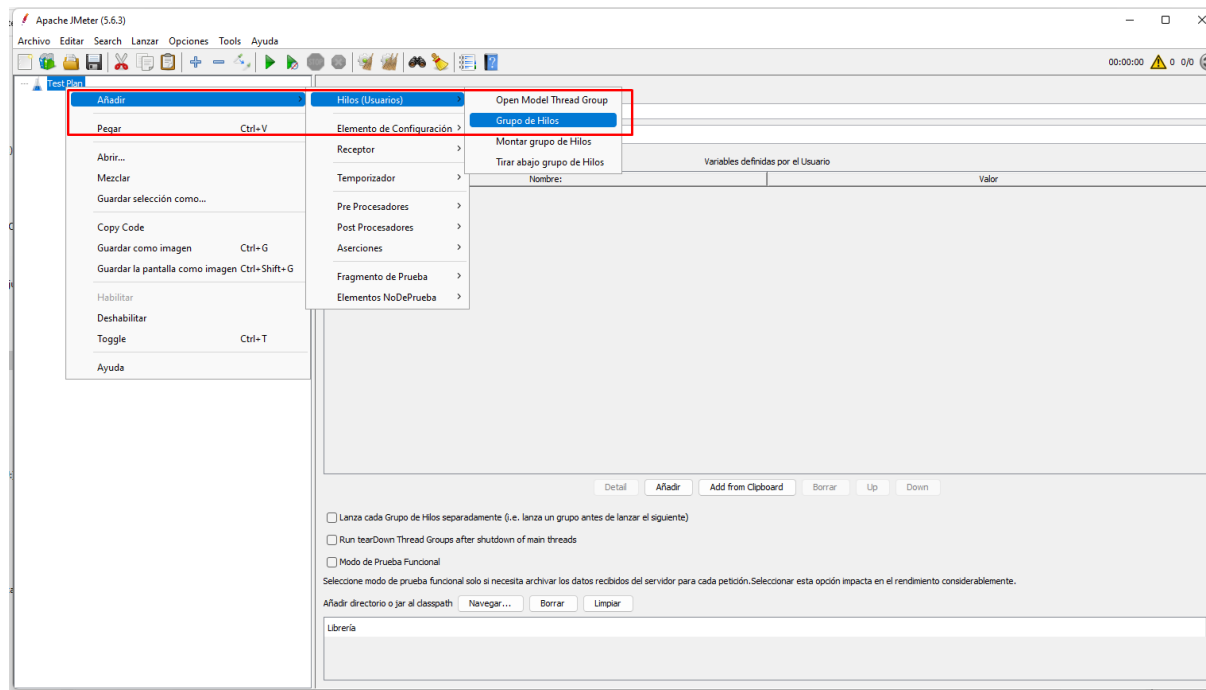
# Instalación

## Entorno de trabajo Jmeter



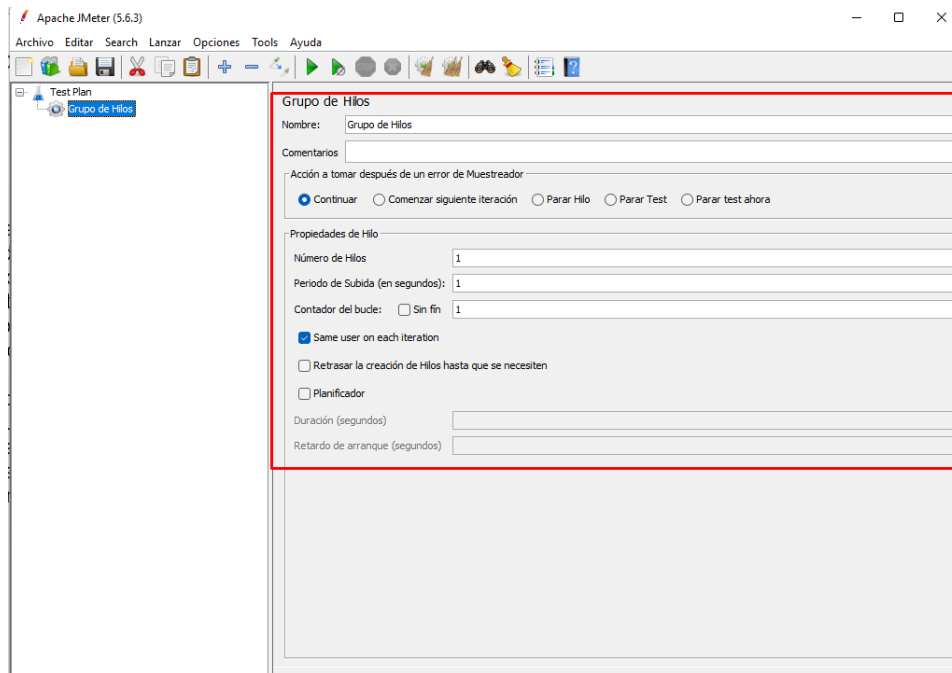
## Prueba de Carga Básica

El componente principal de JMeter es denominado Plan de Prueba o Test Plan, en él se definen todos los aspectos relacionados con una prueba de carga, como: parámetros empleados por requisición, tipo de reportes a generarse con los resultados obtenidos, la posible reutilización de requisiciones compuestas por usuarios, entre otros aspectos.



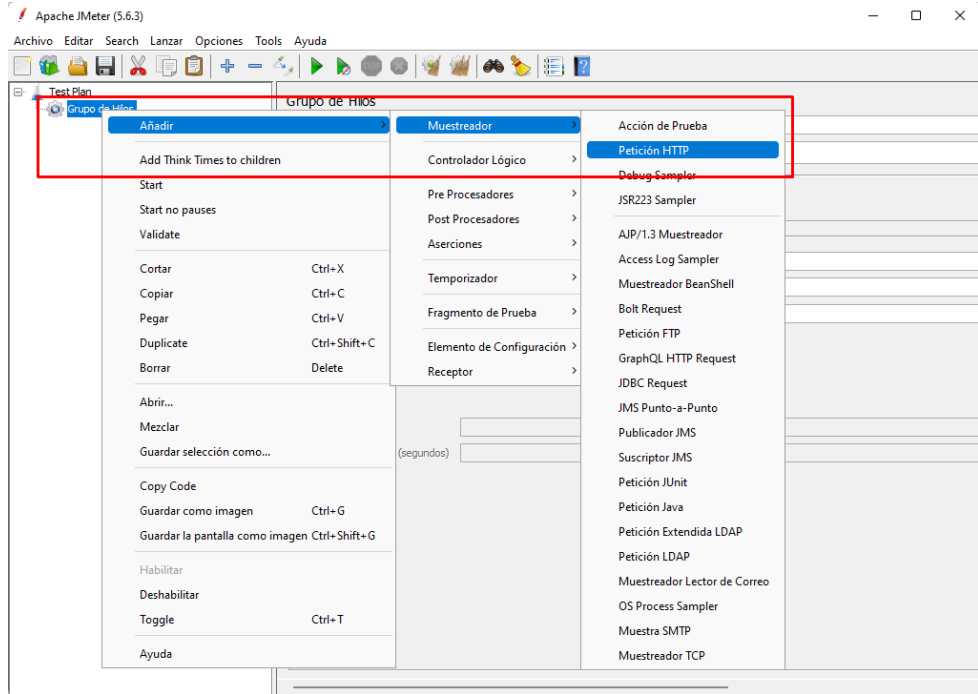
## Prueba de Carga Básica

- **Nombre:** Indica el nombre que se le asignara a la prueba.
- **Número de Hilos:** Número de usuarios que se simulará.
- **Periodo de subida:** Es el lapso de tiempo en segundos que se desea tener entre cada grupo de usuarios.
- **Sin Fin:** Esto es para indicar si la simulación para grupos de usuarios se realizará infinitamente, esto es, si se selecciona esta opción indica que desea simular la cantidad de usuarios ingresada, esperando el tiempo de periodo de subida, luego volver a simular los mismos usuarios por la misma cantidad de tiempo infinitamente. Por lo que esta opción se mantiene desactivada.



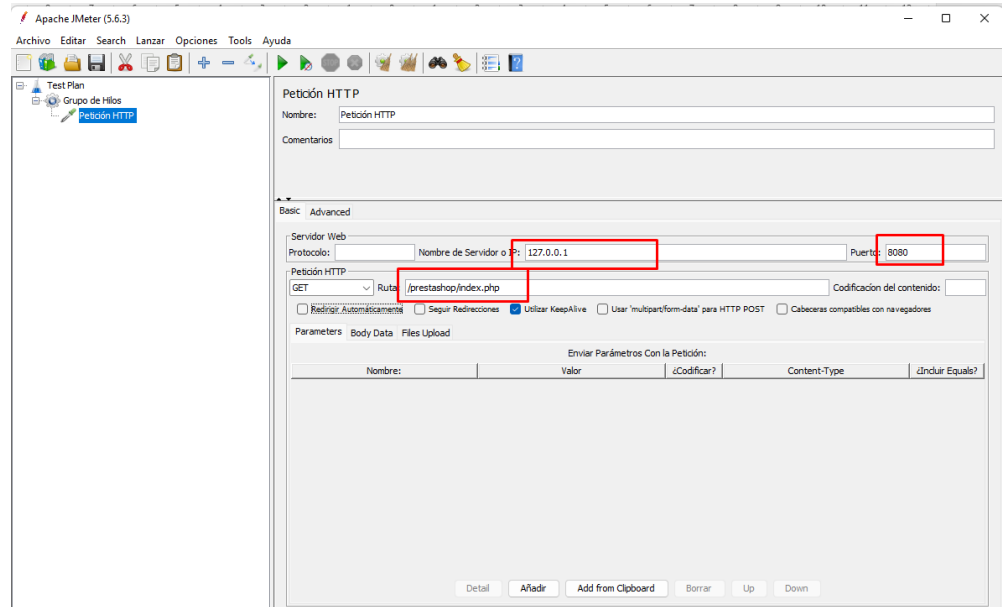
## Prueba de Carga Básica

- Una vez definidas las características del grupo de usuarios se agregará una Petición HTTP. Para ello se accederá desde el *Grupo de Hilos creados, Grupo de Hilos* → *Añadir* → *Muestrador* → *Petición HTTP*.



## Prueba de Carga Básica

- **Nombre de Servidor o IP:** En este caso utilizaremos la dirección IP 127.0.0.1 para trabajar una aplicación local.
- **Puerto:** Define el puerto TCP de operación del servidor, se utilizará el puerto 8080.
- **Ruta:** Utilizado para definir la ruta de acceso para llevar acabo la prueba, será definido /prestashop/index.php.

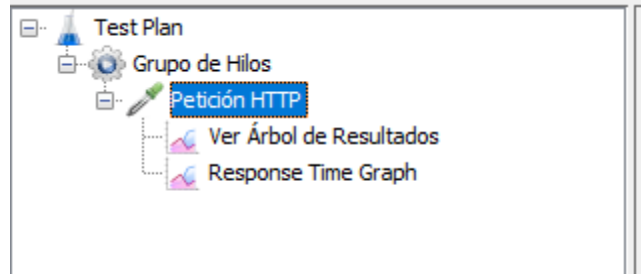




## Prueba de Carga Básica

Para observar los resultados agregaremos dos **receptores** de muestra desde la Petición HTTP:

- Petición HTTP → Añadir → Receptores → Gráfico de Resultados
- Petición HTTP → Añadir → Receptores → Response Time Graph



## Prueba de Carga Básica

Prueba con 5 usuarios  
Ingresamos los siguientes datos.

- Numero de Hilos: 5
- Periodo de Subida: 2
- Contador de bucle: 4

Grupo de Hilos

Nombre: Grupo de Hilos

Comentarios

Acción a tomar después de un error de Muestreador

☒ Continuar ☐ Comenzar siguiente iteración ☐ Parar Hilo ☐ Parar Test ☐ Parar test ahora

Propiedades de Hilo

Número de Hilos: 5

Periodo de Subida (en segundos): 2

Contador del bucle: ☐ Sin fin 4

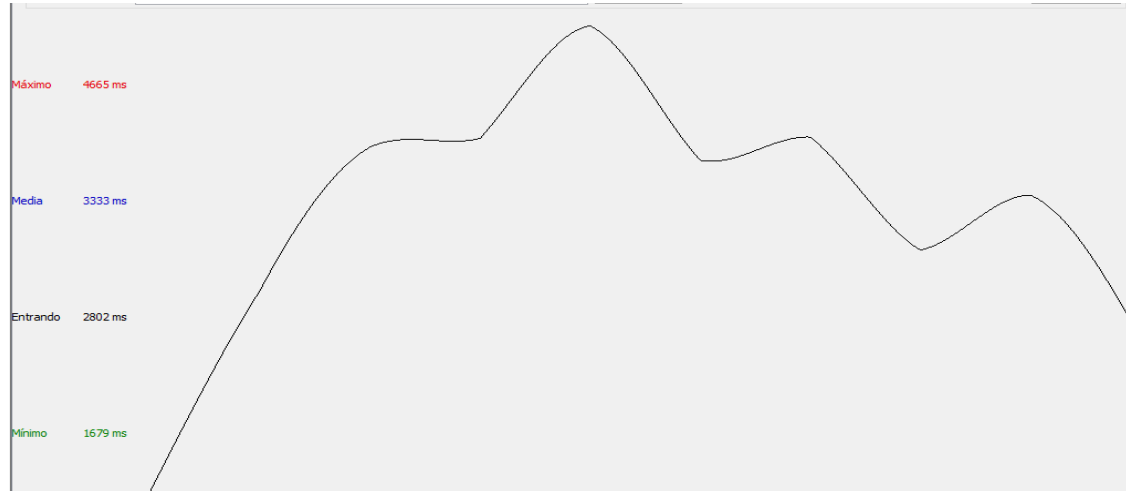
☐ Delay Thread creation until needed

☐ Planificador

Resultados:

| Muestra # | Tiempo de comienzo | Nombre del hilo    | Etiqueta               | Tiempo de Muestra (ms) | Estado                                                                                | Bytes  | Latency | Connect Time(ms) |
|-----------|--------------------|--------------------|------------------------|------------------------|---------------------------------------------------------------------------------------|--------|---------|------------------|
| 1         | 08:27:36.750       | Grupo de Hilos 1-1 | Prueba de petición ... | 1679                   |    | 118848 | 116     | 4                |
| 2         | 08:27:37.139       | Grupo de Hilos 1-2 | Prueba de petición ... | 2012                   |    | 118848 | 136     | 2                |
| 3         | 08:27:37.535       | Grupo de Hilos 1-3 | Prueba de petición ... | 3028                   |    | 118848 | 455     | 2                |
| 4         | 08:27:37.935       | Grupo de Hilos 1-4 | Prueba de petición ... | 2850                   |    | 118858 | 233     | 2                |
| 5         | 08:27:38.432       | Grupo de Hilos 1-1 | Prueba de petición ... | 3184                   |    | 118847 | 905     | 1                |
| 6         | 08:27:38.334       | Grupo de Hilos 1-5 | Prueba de petición ... | 4245                   |    | 118846 | 615     | 3                |
| 7         | 08:27:39.153       | Grupo de Hilos 1-2 | Prueba de petición ... | 4043                   |    | 118847 | 1456    | 1                |
| 8         | 08:27:40.565       | Grupo de Hilos 1-3 | Prueba de petición ... | 3482                   |    | 118849 | 782     | 1                |
| 9         | 08:27:40.786       | Grupo de Hilos 1-4 | Prueba de petición ... | 4665                   |    | 118847 | 1528    | 2                |
| 10        | 08:27:41.619       | Grupo de Hilos 1-1 | Prueba de petición ... | 4080                   |    | 118851 | 1340    | 1                |
| 11        | 08:27:42.581       | Grupo de Hilos 1-5 | Prueba de petición ... | 3720                   |    | 118853 | 1492    | 1                |
| 12        | 08:27:43.197       | Grupo de Hilos 1-2 | Prueba de petición ... | 3567                   |    | 118851 | 807     | 2                |
| 13        | 08:27:44.049       | Grupo de Hilos 1-3 | Prueba de petición ... | 4166                   |    | 118849 | 1764    | 2                |
| 14        | 08:27:45.453       | Grupo de Hilos 1-4 | Prueba de petición ... | 3373                   |    | 118849 | 718     | 1                |
| 15        | 08:27:46.302       | Grupo de Hilos 1-5 | Prueba de petición ... | 2719                   |    | 118851 | 675     | 2                |
| 16        | 08:27:45.701       | Grupo de Hilos 1-1 | Prueba de petición ... | 3595                   |   | 118849 | 1088    | 2                |
| 17        | 08:27:46.765       | Grupo de Hilos 1-2 | Prueba de petición ... | 3943                   |  | 118851 | 1730    | 2                |
| 18        | 08:27:48.217       | Grupo de Hilos 1-3 | Prueba de petición ... | 2964                   |  | 118849 | 493     | 1                |
| 19        | 08:27:48.828       | Grupo de Hilos 1-4 | Prueba de petición ... | 2561                   |  | 118849 | 488     | 1                |
| 20        | 08:27:49.024       | Grupo de Hilos 1-5 | Prueba de petición ... | 2802                   |  | 118845 | 906     | 2                |

## Prueba de Carga Básica



# ¿Qué se logró?



- ✓ Identificar su propósito, beneficios y cuándo deben realizarse en el ciclo de desarrollo.
- ✓ Reconocer las características y objetivos de pruebas de carga, estrés, escalabilidad y estabilidad.
- ✓ Entender y aplicar los conceptos de concurrencia, latencia y throughput para evaluar el comportamiento del sistema bajo diferentes condiciones.

# ¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



## Referencias Bibliográficas

- Proceso de Construcción de Software 2, Mag. Natalí Flores Lafosse, Maestría en Informática de la PUCP, 2018.
- Sommerville, I. (2005). Ingeniería del software. Pearson educación.
- Bourque, P., & Fairley, R. E. (2014). Guide to the software engineering body of knowledge (SWEBOK (R)): Version 3.0. IEEE Computer Society Press.

# GRACIAS